cJSON 多 JSON 字节流解析示例

基于 cJSON 1.7.19

场景：串口 / TCP / MQTT 等字节流通道上，多个不同组合形式的 JSON 字符串连续到达，且可能被拆包（一帧分多次到达）或粘包（一次到达多帧）。

本文给出完整的拼帧 + 队列 + 解析实现。

第 3~6 节的代码块按顺序合并即为一个完整 .c 文件，可直接编译：

gcc 示例.c cJSON.c -o 示例（所有用到的函数与变量在各节中均有定义）。

目录

1. 问题模型
2. 总体方案
3. 帧队列实现
4. 括号配对拼帧器
5. 按数据组合方式分派解析
6. 完整演示：多组合帧的拆包与粘包
7. 工程注意事项

1. 问题模型

发送方连续发出 4 个不同组合形式的 JSON 帧：

```
{"cmd":"led","val":1}[1,2,3]{"user":{"name":"Tom","tags":["a","b"]}}{"note":"{含括号\"}"}
```

接收方实际可能这样到达：

```
第1次 recv: {"cmd":"led","val":1}[1,2
第2次 recv: ,3]{"user":{"name":"Tom","ta
第3次 recv: gs":["a","b"]}}{"note":"{含括号\"}"}
```

现象	说明
拆包	一帧被拆成多次到达，绝不能收到就 Parse
粘包	一次到达含多个完整帧，需逐帧切分
字符串陷阱	字符串内容里可能有 } { \"，不能作为帧边界

2. 总体方案

```
字节流 ——→ [括号配对拼帧器] ——→ [帧队列] ——→ [消费者: Parse → 按类型分派 → Delete]
        逐字节状态机      缓存完整帧       固定四步
```

帧边界采用括号配对法（对端协议不可改时的通用方案）：

- 遇 { / [深度 +1，遇 } /] 深度 -1，深度归 0 即一帧结束
- 字符串内部（"..."）的括号不参与计数，并处理 \" 转义

3. 帧队列实现

```
/* ==================== 第 1 部分：帧队列（链表实现） ==================== */
#include &lt;stdlib.h&gt;   /* malloc / free / realloc */
#include &lt;string.h&gt;   /* memcpy / memset / strlen */
#include &lt;stdio.h&gt;    /* printf */
#include "cJSON.h"

typedef struct jnode {
    char        *data;    /* 完整 JSON 帧（补 '\0'） */
    size_t       len;
    struct jnode  *next;
} jnode_t;

typedef struct {
    jnode_t *head, *tail;
    int     count;
    int     max;         /* 队列上限，防积压 */
} json_queue_t;

void json_queue_init(json_queue_t *q, int max)
{
    q-&gt;head = q-&gt;tail = NULL;
    q-&gt;count = 0;
    q-&gt;max = max;
}

int json_queue_push(json_queue_t *q, const char *frame, size_t len)
{
    jnode_t *n;
    if (q-&gt;count &gt;= q-&gt;max) return -1;
    n = (jnode_t *)malloc(sizeof(jnode_t));
    if (n == NULL) return -1;
    n-&gt;data = (char *)malloc(len + 1);
    if (n-&gt;data == NULL) { free(n); return -1; }
    memcpy(n-&gt;data, frame, len);
    n-&gt;data[len] = '\0';          /* 保证可直接给 cJSON_Parse */
    n-&gt;len = len; n-&gt;next = NULL;
    if (q-&gt;tail) q-&gt;tail-&gt;next = n; else q-&gt;head = n;
    q-&gt;tail = n;
    q-&gt;count++;
    return 0;
}

char *json_queue_pop(json_queue_t *q)   /* 返回 malloc 字符串，调用者 free */
{
    jnode_t *n = q-&gt;head;
    char *data;
    if (n == NULL) return NULL;
    q-&gt;head = n-&gt;next;
    if (q-&gt;head == NULL) q-&gt;tail = NULL;
    q-&gt;count--;
    data = n-&gt;data;
    free(n);
    return data;
}

void json_queue_clear(json_queue_t *q)
{
    char *f;
    while ((f = json_queue_pop(q)) != NULL) free(f);
}
```

4. 括号配对拼帧器

```
/* ==================== 第 2 部分：括号配对拼帧器（状态机） ==================== */
#define JSTREAM_BUF_INIT  256        /* 拼帧缓冲初始容量 */
#define JSTREAM_BUF_MAX   (64 * 1024)  /* 单帧上限，超限丢帧重同步 */

typedef struct {
    char  *buf;
    size_t len, cap;
    int    depth;   /* 括号配对深度 */
    int    started; /* 是否已遇帧起始 { 或 [ */
    int    in_str;  /* 是否在 "..." 内 */
    int    esc;     /* 上一字符是否 \ */
} jstream_t;

void jstream_init(jstream_t *s)
{
    memset(s, 0, sizeof(*s));
    s-&gt;cap = JSTREAM_BUF_INIT;
    s-&gt;buf = (char *)malloc(s-&gt;cap);
}

void jstream_reset(jstream_t *s)
{
    s-&gt;len = 0; s-&gt;depth = 0; s-&gt;started = 0; s-&gt;in_str = 0; s-&gt;esc = 0;
}

void jstream_free(jstream_t *s) { free(s-&gt;buf); memset(s, 0, sizeof(*s)); }

/* 喂入一段字节流；每凑齐一帧推入队列（一次可产出 0~N 帧） */
void jstream_feed(jstream_t *s, const char *data, size_t n, json_queue_t *q)
{
    size_t i;
    for (i = 0; i &lt; n; i++) {
        char c = data[i];

        if (s-&gt;len + 1 &gt;= s-&gt;cap) {              /* 缓冲扩容 */
            if (s-&gt;cap &gt;= JSTREAM_BUF_MAX) { jstream_reset(s); continue; }
            s-&gt;cap *= 2;
            s-&gt;buf = (char *)realloc(s-&gt;buf, s-&gt;cap);
        }

        if (!s-&gt;started) {                      /* 跳过帧前噪声 */
            if (c == '{' || c == '[') {
                s-&gt;started = 1; s-&gt;depth = 1;
                s-&gt;buf[0] = c;  s-&gt;len = 1;
            }
            continue;
        }

        s-&gt;buf[s-&gt;len++] = c;

        if (s-&gt;in_str) {                        /* 字符串内括号不计数 */
            if (s-&gt;esc)        s-&gt;esc = 0;
            else if (c == '\\')  s-&gt;esc = 1;
            else if (c == '"')   s-&gt;in_str = 0;
            continue;
        }

        switch (c) {
        case '"':  s-&gt;in_str = 1; break;
        case '{':
        case '[':  s-&gt;depth++;    break;
        case '}':
        case ']':
            if (--s-&gt;depth == 0) {              /* 凑齐一帧 */
                json_queue_push(q, s-&gt;buf, s-&gt;len); /* 队列满则丢帧 */
                jstream_reset(s);
            }
            break;
```

```
    default: break;
    }
  }
}
```

5. 按数据组合方式分派解析

收到的帧可能是对象、数组、嵌套混合等任意组合，消费者先判根类型再分派：

```
/* ==================== 第 3 部分：消费者（按根类型分派解析） ==================== */
/* 处理对象帧：{"cmd":"led","val":1} 或嵌套 {"user":{...}} */
static void handle_object(cJSON *root)
{
  cJSON *cmd = cJSON_GetObjectItem(root, "cmd");
  if (cJSON_IsString(cmd)) {
    printf("[对象帧] cmd=%s", cmd-&gt;valuestring);
    cJSON *val = cJSON_GetObjectItem(root, "val");
    if (cJSON_IsNumber(val)) printf(" val=%g", val-&gt;valuedouble);
    printf("\n");
    return;
  }
  /* 嵌套对象：遍历第一层键值 */
  cJSON *kv = NULL;
  cJSON_ArrayForEach(kv, root) {
    printf("[对象帧] 键=%s 类型=%d\n", kv-&gt;string, kv-&gt;type &amp; 0xFF);
  }
  /* 嵌套取值演示：user.tags[1] */
  cJSON *user = cJSON_GetObjectItem(root, "user");
  cJSON *tags = user ? cJSON_GetObjectItem(user, "tags") : NULL;
  cJSON *t1   = tags ? cJSON_GetArrayItem(tags, 1) : NULL;
  if (cJSON_IsString(t1)) printf("  user.tags[1]=%s\n", t1-&gt;valuestring);
}

/* 处理数组帧：[1,2,3] 或 [{"id":1},...] */
static void handle_array(cJSON *root)
{
  int n = cJSON_GetArraySize(root);
  printf("[数组帧] 长度=%d 元素:", n);
  cJSON *e = NULL;
  cJSON_ArrayForEach(e, root) {
    if (cJSON_IsNumber(e))      printf(" %g", e-&gt;valuedouble);
    else if (cJSON_IsObject(e)) printf(" {对象}");
    else if (cJSON_IsString(e)) printf(" \"%s\"", e-&gt;valuestring);
  }
  printf("\n");
}

/* 消费者：出队 → 解析 → 分派 → 释放 */
void json_worker(json_queue_t *q)
{
  char *frame;
  while ((frame = json_queue_pop(q)) != NULL) {
    cJSON *root = cJSON_Parse(frame);
    printf("---- 帧: %s\n", frame);
    free(frame);                  /* 帧文本先用完即放 */
    if (root == NULL) {
      printf("解析失败: %s\n", cJSON_GetErrorPtr());
      continue;
    }
    if     (cJSON_IsObject(root)) handle_object(root);
    else if (cJSON_IsArray(root))  handle_array(root);
    else printf("[其他类型] type=%d\n", root-&gt;type &amp; 0xFF);
    cJSON_Delete(root);           /* 树结构统一释放 */
  }
}
```

6. 完整演示：多组合帧的拆包与粘包

```
/* ==================== 第 4 部分：演示 main ==================== */
int main(void)
{
  json_queue_t q; json_queue_init(&amp;q, 16);  /* 队列最多缓存 16 帧 */
  jstream_t   js; jstream_init(&amp;js);

  /* 发送方原始字节流（4 个不同组合的帧连发）：
    帧1 简单对象:  {"cmd":"led","val":1}
    帧2 纯数组:    [1,2,3]
    帧3 嵌套对象:  {"user":{"name":"Tom","tags":["a","b"]}}
    帧4 含陷阱字符: {"note":"}{含括号\"}"}                */

  /* 模拟拆包 + 粘包：分 3 次到达 */
  const char *rx1 = "{\"cmd\":\"led\",\"val\":1}[1,2";
  const char *rx2 = ",3]{"                  \
                "\"user\":{\"name\":\"Tom\",\"ta";
  const char *rx3 = "gs\":[\"a\",\"b\"]}}{\"note\":\"}{含括号\\\"}\"}";

  jstream_feed(&amp;js, rx1, strlen(rx1), &amp;q);  /* 产出帧1，帧2收一半 */
  jstream_feed(&amp;js, rx2, strlen(rx2), &amp;q);  /* 产出帧2，帧3收一半 */
  jstream_feed(&amp;js, rx3, strlen(rx3), &amp;q);  /* 产出帧3、帧4 */

  json_worker(&amp;q);

  jstream_free(&amp;js);
  json_queue_clear(&amp;q);
  return 0;
}
```

预期输出：

```
---- 帧: {"cmd":"led","val":1}
[对象帧] cmd=led val=1
---- 帧: [1,2,3]
[数组帧] 长度=3 元素: 1 2 3
---- 帧: {"user":{"name":"Tom","tags":["a","b"]}}
[对象帧] 键=user 类型=64
  user.tags[1]=b
---- 帧: {"note":"}{含括号\"}"}
[对象帧] 键=note 类型=16
```

演示覆盖的数据组合：

帧	组合形式	考察点
帧1	简单对象	与帧2粘包（一次 recv 含两帧边界）
帧2	纯数字数组	根节点为 [开头，跨包补齐
帧3	三层嵌套对象含数组	深度配对正确性，嵌套路径取值
帧4	字符串内含 } { \"	拼帧器最易踩的坑：字符串内括号不计数

7. 工程注意事项

事项	说明
队列上限	json_queue_init(&q, 16) 限制缓存帧数，消费慢时丢帧保护内存
单帧上限	JSTREAM_BUF_MAX 防畸形数据撑爆内存，超限自动重新同步
线程安全	生产者（中断/接收线程）与消费者（主循环/工作线程）并发时，push/pop 需加锁或关中断

事项	说明
裸机环境	可将链表队列换成环形缓冲 + 固定帧槽，避免 malloc
配对类型	状态机只数深度不校验 { 与] 是否匹配，残次帧由 cJSON_Parse 兜底拒绝
释放顺序	消费者固定四步：pop → Parse → free(frame) → cJSON_Delete(root)
已知长度场景	队列中保存了 len，也可改用 cJSON_ParseWithLength(frame, len) 而不依赖 \0
协议可自定义	更简单的替代：长度前缀法 [4字节长度][JSON]，或换行分隔法（JSON 需无裸换行，可用 cJSON_PrintUnformatted 发送）